

RISC-V TPU 指令集设计文档

修改记录

版本	修改人	时间	备注
V1.0	孟成真	20260430	

1. 寄存器设计

本章定义 RISC-V Tensor 指令集扩展中的软件可见寄存器状态，包括 Tensor 寄存器文件、Temp 寄存器、Mask 寄存器以及相关控制状态寄存器。除特别说明外，本章中的“Tensor 寄存器”均指每个 hart 私有的核内 Tensor local memory 资源，而不是 RISC-V 通用寄存器。

Tensor 扩展采用“本地 Tensor 寄存器文件 + 标量寄存器动态描述操作数”的设计方式。每个 hart 拥有一组本地 Tensor line。Tensor 指令不直接在指令编码中固定 Tensor 寄存器编号，而是通过标量寄存器传递 Tensor 操作数描述符。

1.1 Tensor 寄存器文件

Tensor 寄存器按照 Tensor Line 组织，使用从 0 开始的线性地址索引，每个 Tensor Line 的宽度为 $TWID \times THEI$ 。在当前版本的设计中，取 $TWID = THEI = 32$ ，即每个 Tensor Line 的宽度为： $32 \times 32 \times 8 \text{ bit} = 8192 \text{ bit} = 1024 \text{ Byte}$ 。通过读取 tregnum CSR 获取当前机器的逻辑 Tensor Line 数量。Tensor Line 数量最大值 1024。

Tensor 指令的基础操作单元是 Tile。当前版本仅支持 Row-Major 编码 32×32 Tile。在不同的 Tile 尺寸和元素宽度下，一个逻辑 Tile 对应若干个连续的逻辑 Tensor Line，映射关系如下：

元素宽度	Tile 尺寸	占用 Tensor Line	索引对齐	映射
8 bits	32×32	1	1-line 对齐	TR[n]: 目标 Tile
16 bits	32×32	2	2-line 对齐	TR[n+0]: 每个元素的低 8 位 TR[n+1]: 每个元素的高 8 位
32 bits	32×32	4	4-line 对齐	TR[n+0]: 每个元素的 0-7 位 TR[n+1]: 每个元素的 8-15 位 TR[n+2]: 每个元素的 16-23 位 TR[n+3]: 每个元素的 24-31 位

1.2 Temp 寄存器

Temp 寄存器用于保存短生命周期中间 Tile 结果，主要服务于 GEMM 后处理和链式计算，宽度为最大可用的 Tile 尺寸。Temp 仅能用于 GEMM 累加器读写指令的操作数、逐元素算术指令的 TRS1 和 TRD 操作数。

Temp 仅包含一个逻辑寄存器。Temp 寄存器本身不携带类型信息，对 Temp 寄存器的读写语义与 Tensor 寄存器一样通过 Tensor 描述符进行。Tensor 指令对 Temp 的读写与普通 Tensor line 一样参与依赖顺序检查。

1.3 Mask 寄存器

Mask 是 Tensor 扩展中的按元素掩码寄存器，其尺寸为 $32 \times 32 \times 1$ bit。每个 bit 按照 Row-Major 编码对应 Tile 中的元素。

默认的，Mask 寄存器的功能是作为 Tensor 指令的写回掩码，无效掩码对应的元素在写回时被置零，并不是像 V 指令一样保持原值；特殊的，在 Tensor Store 指令和 Tensor Reduce 系列指令中作为源元素的有效掩码，在 Tensor Mask 系列指令和 Tensor Compare 系列指令中作为源操作数或目标操作数。

Mask 寄存器仅对设置 Msk bit 为 1 的指令生效。

1.4 Acc 隐式累加寄存器

Acc 是 Tensor GEMM 指令隐式使用的寄存器，保存连续 GEMM 过程中的累加结果。Acc 寄存器仅能通过 Tensor GEMM 指令使用，不能作为 Tensor 操作数。

1.5 Tensor 操作数描述符编码

Tensor 指令不直接编码完整 Tensor 寄存器编号、数据类型和属性。相反，

软件通过标量寄存器传递 Tensor 操作数描述符。每个 Tensor 操作数描述符占据 32 bits 宽度，一个 RV64 标量寄存器可以传递最多两个 Tensor 操作数描述符。

Tensor 操作数描述符格式如下：

31:19	18	17	16	15:10	9:0
Reserved	IsZero	IsMask	IsTemp	DType	Index

其中：

- Index：起始 Tensor line 索引编号
- DType：元素数据类型
- IsTemp：是否使用 Temp 寄存器，覆盖 Index 值
- IsMask：是否使用 Mask 寄存器，覆盖 Index 值
- IsZero：是否使用全 0 值，覆盖 Index 值

DType 编码如下：

DType[5:4]	DType[3:0]	数据类型	是否必须实现
0	0	SINT8	Y
0	1	UINT8	Y
0	2	FP8 E4M3	tcap[0]
0	3	FP8 E5M2	tcap[1]
0	4	FP8 E3M4	tcap[2]
1	0	SINT16	Y
1	1	UINT16	Y
1	2	IEEE FP16	Y
1	3	BF16	Y
2	0	INT32 算术/累加	Y
2	0	INT32 矩阵乘	tcap[3]
2	1	UINT32	tcap[4]
2	2	IEEE FP32 算术/累加	Y
2	2	IEEE FP32 矩阵乘	tcap[5]
2	3	BF32	tcap[6]
2	4	TF32	tcap[7]
3	0	SINT4	tcap[8]

3	1	UINT4	tcap[9]
3	2	FP4 E2M1	tcap[10]

2. CSR 与异常处理

2.1 CSR 总览

Tensor 扩展建议定义以下 CSR：

CSR 名称	属性	说明
tinfo	RO	Tensor 扩展版本和基本信息
tregnum	RO	逻辑 Tensor line 数量
tcap	RO	支持的可选数据类型集合
tslotnum	RO	Tensor FIFO Slot 个数
texpt	RW	从上次清除后第一次 Tensor 非精确异常信息
tepc	RO	texpt 对应的异常发生时的 PC
tepos	RO	texpt 对应的异常元素在 Tile 中的位置
tval	RO	texpt 对应的异常携带的参数
tdirty0-15	RW	Tensor 寄存器中所有被修改的 Tensor Line 的 Bitmap，每个 CSR 表示 64 Tensor Lines。超出 Tensor Line 总数的位置始终为 0
tdirtyl2	RW	tdirty0-15 CSR 是否非 0 的 Bitmap。对其置位会同时设置对应 tdirty CSR 中的所有位

2.2 Tensor 异常处理

Tensor 异常分为两类：

- 精确异常：发生在指令译码与操作数检查阶段，按照普通 RISC-V 异常机制处理；
- 非精确异常：发生在 Tensor 指令执行阶段，仅由 Tensor Load、Store、Scatter、Gather 指令的访存失败引起，通过 Tensor 异常 CSR 记录。

2.2.1 Tensor 精确异常

精确异常是指在 Tensor 指令进入 Tensor 执行单元之前即可确定的异常。此类异常发生在 CPU 前端、译码阶段、权限检查阶段或 Tensor 操作数合法性检查阶段。

精确异常应按照 RISC-V 标准异常机制处理。也就是说，处理器应更新相应的异常 CSR，并跳转到异常处理入口。发生精确异常的 Tensor 指令不应被提交到 Tensor 执行单元中执行。

Tensor 指令可能产生以下精确异常，除了取值异常外，其他 Tensor 精确异常均为非法指令异常：

异常来源	说明
指令取值异常	获取 Tensor 指令本身时发生 instruction access fault 或 instruction page fault
Tensor 越界	Tensor line 地址超过 tregnum 所定义范围
Tensor 未对齐	16-bit 操作数未 2-line 对齐，32-bit 操作数未 4-line 对齐
不支持的数据类型	使用硬件未声明支持的 dtype
不合法的 Temp/TMask 使用	在不允许的指令位置使用 Temp 或 TMask
不合法的指令字段组合	例如 GEMM 中 A/B dtype 不一致

2.2.2 非精确异常

非精确异常是指 Tensor 指令已经被提交到 Tensor 执行单元，并在执行阶段发生的异常。仅 TLOAD, TSTORE, TGATHER, TSCATTER 指令会在访存阶段产生非精确异常。其他所有指令不产生任何执行期异常。

Tensor 非精确异常通过 Tensor 状态 CSR 中的异常 flag 记录。包含以下 flag：

异常名	说明
TEXPT_ACCESS_FAULT	Tensor 访存异常
TEXPT_PAGE_FAULT	Tensor 页面错误

TEXPT_ACCESS_FAULT 表示 Tensor Load/Store/Scatter/Gather 在访问主存时发生访问错误，例如物理内存访问失败、总线错误、权限外访问或 PMP/PMA 相关访问失败。TEXPT_PAGE_FAULT 表示 Tensor Load/Store/Scatter/Gather 在地址翻译过程中发生页面错误，例如页表项无效、

权限不足、缺页或地址翻译失败。

Tensor 非精确异常不触发 RISC-V trap。发生非精确异常时，硬件只设置 `texpt` `tepc` `tval` `tepos` CSR。软件需要在合适位置显式读取 Tensor 异常 CSR，并判断 Tensor 计算流程是否成功完成。当 Tensor 访存指令发生非精确异常时，该指令及其之后相关 Tensor 指令的结果状态可能是不完整的。

3. Tensor 指令编码总览

本扩展使用 RISC-V custom opcode 空间定义 TPU Tensor 指令。基础编码采用 32-bit 指令长度，opcode 0001011。Tensor 指令的语义参数主要由标量寄存器动态提供，指令编码本身主要提供操作类别、控制位和少量静态修饰字段。这种方式允许软件运行时动态管理 Tensor local memory，并支持 Tensor 指令在提交阶段拼接成完整 TPU 微操作。

Tensor 指令的一般编码结构如下：

31	30:25	24:20	19:15	14:12	11:7	6:0
mask	funct6	rs2	rs1	group	sub-op	opcode

后续文档使用如下记号表示 Tensor 操作数描述符：

记号	含义
TRD	Tensor 目的操作数描述符
TRS1	Tensor 源操作数 1 描述符
TRS2	Tensor 源操作数 2 描述符
XXX-T	可指向 Temp 寄存器的描述符
XXX-M	可指向 Mask 寄存器的描述符
XXX-8	仅能使用拥有 8 bits 宽度数据类型的描述符

后续文档中，对于指令名，采用两种不同的表示用于区别指令编码中携带的信息和通过标量寄存器传递的信息，如：

TLOAD[.Row|.Col][.M][.Sn] TRD, BaseAddr, [Stride]

其中指令名带有的可选后缀表示由指令本身编码的编译时信息，而后面跟的操作数 TRD 和值 BaseAddr 与 Stride 则是通过标量寄存器传递的信息。

4. Tensor 访存指令

Tensor Memory 指令负责在主存与 Tensor local memory 之间搬运一个 32×32 Tensor Tile。该组包括连续或规则布局访问的 Load/Store，也包括以元素偏移表驱动的 Gather/Scatter。

对于所有的 Tensor Memory 指令，不同元素的访存顺序不被保证。任意元素访存失败均会设置异常 CSR。如果一条指令中多个元素均引起异常，最终写入异常 CSR 的异常参数和索引位置为其中任意一个。

(注：目前的版本中虽然指定了 Page Fault 异常，但建议实现时不启用虚拟内存机制，或仅启用有限的物理内存保护机制。本指令集对于虚拟内存和上下文切换的适配仍需完善。)

4.1 Tensor 地址对齐规则

所有的 Tensor 访存基地址 BaseAddr 必须对齐到目标 Tile 的主序数据宽度。在 $TWID = THEI = 32$ 的设置下，BaseAddr 的对齐要求如下：

元素宽度	BaseAddr 对齐
8 bits	32 Bytes
16 bits	64 Bytes
32 bits	128 Bytes

注：后续如果支持行列数不相等的 Tile 配置，TLOAD.Col/TSTORE.Col 的对齐规则会不同于其他访存指令。

4.2 TLOAD[.Row|.Col][.M][.Sn] TRD, BaseAddr, [Stride]

4.3 TSTORE[.Row|.Col][.M][.Sn] TRS1, BaseAddr, [Stride]

TLOAD 从主存地址 BaseAddr 开始加载一个 Tensor Tile，并写入目的 Tensor 操作数 TRD。TSTORE 将源 Tensor Tile 写回主存中以地址 MemBaseAddr

开始的位置。可选编码如下：

编码	说明
Row	按照行主序布局加载，Stride 表示列元素偏移个数，默认为连续加载 Tile
Col	按照列主序布局加载，Stride 表示行元素偏移个数，默认为连续加载 Tile
M	启用 Mask，跳过 Mask 寄存器中标记为 0 的元素（置 0），默认为不跳过
Sn	连续加载的元素之间间隔的元素个数(0-15)，默认为 0

4.4 TGATHER[.M] TRD, TRS2, BaseAddr, RowSel

4.5 TSCATTER[.M] TRS1, TRS2, BaseAddr, RowSel

TGATHER 根据 offset Tensor TRS2 对主存执行按行 gather 访问，从多个不连续位置将每一行数据加载到目的 Tensor Tile TRD。TSCATTER 根据 offset Tensor 将源 Tensor Tile TRS1 中的每一行数据写入主存中的不同地址。语义上：

$$\text{Addr}[r][c] = [\text{BaseAddr} + (\text{TRS2}[\text{RowSel}][r] + c) * \text{sizeof}(\text{dtype})]$$

当 Msk = 1 时，Mask 掩码对 TRS2 生效，仅对 mask[RowSel][r] 选中的行执行 gather 或 scatter。对于 TGATHER，未选中的行会被置 0。对于 TSCATTER，未选中的行不产生任何内存写入。

4.6 TGATHERB(.Bn)[.M] TRD, TRS2, BaseAddr, Row, Col

4.7 TSCATTERB(.Bn)[.M] TRS1, TRS2, BaseAddr, Row, Col

TGATHERB 按照 Tile Block 为单位从多个不连续地址加载多个 Tile Block 并拼接为一个 Tensor Tile TRD。TSCATTERB 将源 Tensor Tile TRS1 拆分成若干 Tile Block 并加载到不连续的内存地址上。

Tile Block 是在二维上对 Tensor Tile 的更小划分，允许 2x2, 4x4, 8x8, 16x16 尺寸：

BShape[3:0]	n	Tile Block 尺寸	单个 Tile 中 Tile Block 索引个数
1	2	2x2	16x16

2	4	4x4	8x8
4	8	8x8	4x4
8	16	16x16	2x2

语义上：

$$\text{BlockSize} = \text{sizeof}(\text{dtype}) * n * n$$

$$\text{BlockAddr}[\text{br}][\text{bc}] = \text{BaseAddr} + \text{TRS2}[\text{Row} + \text{br}][\text{Col} + \text{bc}] * \text{BlockSize}$$

$$\text{Addr}[\text{br}][\text{bc}][\text{r}][\text{c}] = \text{BlockAddr}[\text{br}][\text{bc}] + \text{sizeof}(\text{dtype}) * (\text{r} * n + \text{c})$$

当 $\text{Msk} = 1$ 时，Mask 掩码对 TRS2 生效，仅对 $\text{mask}[\text{Row} + \text{br}][\text{Col} + \text{bc}]$ 选中的 Tile Block 执行访存。对于 TGATHERB，未选中的 Tile Block 会被置 0。对于 TSCATTERB，未选中的 Tile Block 不会产生访存行为。

4.8 TGATHERRB(.Bn)[.M] TRD, TRS2, BaseAddr, RowSel

4.9 TSCATTERRB(.Bn)[.M] TRS1, TRS2, BaseAddr, RowSel

与 TGATHERB、TSCATTERB 不同的地方在于，TGATHERRB、TSCATTERRB 的 Tile Block 地址偏移在 TRS2 中按行排列。语义上：

$$\text{BlockSize} = \text{sizeof}(\text{dtype}) * n * n$$

$$\text{BlkIdx} = \text{br} * \text{BColNum} + \text{bc}$$

$$\text{BlockAddr}[\text{br}][\text{bc}] = \text{BaseAddr} + \text{TRS2}[\text{RowSel} + \text{BlkIdx} / \text{TWID}][\text{BlkIdx} \% \text{TWID}] * \text{BlockSize}$$

$$\text{Addr}[\text{br}][\text{bc}][\text{r}][\text{c}] = \text{BlockAddr}[\text{br}][\text{bc}] + \text{sizeof}(\text{dtype}) * (\text{r} * n + \text{c})$$

当 $\text{Msk} = 1$ 时，Mask 掩码对 TRS2 生效。对于 TGATHERRB，未选中的 Tile Block 会被置 0。对于 TSCATTERRB，未选中的 Tile Block 不会产生访存行为。

5. Tensor GEMM 指令

所有 TGEMM 指令隐含了一个逻辑 Acc 累加寄存器。

5.1 TGEMM[.TA][.TB][.ACC][.SAT][.EF] TRS1, TRS2

TGEMM 指令对两个 Tensor Tile （TRS1 对应操作数 A，TRS2 对应操作数 B）执行矩阵乘法 $C = TRS1 * TRS2$ ，并将结果写入或累加到 Acc 寄存器中。可选编码如下：

编码	说明
TA	对 TRS1 采用转置输入视图，默认为不转置
TB	对 TRS2 采用转置输入视图，默认为不转置
ACC	将结果矩阵累加到 Acc 寄存器，默认为写入到 Acc 寄存器
SAT	将累加结果饱和到逻辑累加格式的最大/最小有限值，默认为正常溢出
EF	使用精确浮点，仅对 FP8 E4M3/E3M4/E5M2, FP16 源操作数生效。

TGEMM 要求 TRS1 与 TRS2 的 dtype 一致。对于不同的输入 dtype，Acc 寄存器采用的逻辑累加格式对应表如下：

TRS1/TRS2 dtype	Acc 逻辑累加格式
INT8, UINT8, INT16, UINT16	INT32 Exact
INT32	INT32 AccLast（未来扩展）
FP8 E4M3/E3M4/E5M2, FP16	IEEE FP32 Exact or AccLast
BF16	IEEE FP32 AccLast
FP32, BF32, TF32	IEEE FP32 AccLast（未来扩展）

逻辑累加格式定义了累加寄存器在架构可见层面的数值格式。对于 GEMM 指令的累加语义，支持两种精度模式：Exact 和 AccLast。

Exact 精度

在 Exact 精度模式下，累加寄存器在内部保存跨指令延续的精确累加状态。对同一个累加寄存器执行的多条 GEMM 累加指令，其乘积项应被并入该寄存器的内部精确累加状态中，且在指令边界处不得因转换到逻辑累加格式而引入

舍入误差。

设累加寄存器 Acc 的内部精确状态为 $EAcc$ ，逻辑累加格式为 $Tacc$ 。对于一条 GEMM 指令生成的某个输出元素，其输入向量为 $A[0..K-1]$ 和 $B[0..K-1]$ ，则该指令对 $EAcc$ 的更新语义为：

$$EAcc = EAcc + \sum \{ \text{exact}(A[k]) * \text{exact}(B[k]) \}$$

$Exact$ 精度下的舍入只发生在累加寄存器被架构可见读取时。当累加寄存器作为架构可见结果被读取、写回到非 $Exact$ 目的地时，其结果为：

$$\text{Read}(Acc) = Tacc(EAcc)$$

$Exact$ 精度存在有效乘累加次数上限，即对于不同的数据类型，硬件仅在特定次乘累加之内保证 $Exact$ 精度。超出给定次数的连续乘累加产生的溢出属于未定义行为。不同源操作数类型对应的 $Exact$ 精确乘累加次数上限如下：

TRS1/TRS2 dtype	逻辑累加格式	精确乘累加次数	对应 GEMM 指令数
INT8, UINT8	INT32 Exact	65536	2048
INT16, UINT16	INT32 Exact	8192	256
FP8 E4M3/E3M4	FP32 Exact	32768	1024
FP8 E5M2	FP32 Exact	16384	512
FP16	FP32 Exact	8192	256

AccLast 精度

在 $AccLast$ 精度模式下，累加寄存器仅保存逻辑累加格式 $Tacc$ 的架构值。每条 GEMM 指令独立地产生一个指令内局部部分和；该部分和按照目标逻辑累加格式进行顺序乘加，随后再与指令执行前的旧累加值相加。每一次乘加内部以精确乘积和精确加法定义，但每一次乘加结果都舍入到逻辑累加格式 $Tacc$ 。

设旧累加值为：

$$acc_0 = Tacc(Acc_old)$$

对于一条 GEMM 指令生成的某个输出元素，先定义指令内局部部分和：

$$p_0 = 0$$

$$p_{\{k+1\}} = \text{Tacc}(\text{exact}(p_k) + \text{exact}(A[k]) \times \text{exact}(B[k])), \text{ for } k = 0 \dots K-1$$

然后将该部分和与旧累加值相加，得到新的累加寄存器值：

$$\text{Acc_new} = \text{Tacc}(\text{exact}(p_K) + \text{exact}(\text{acc_0}))$$

因此，AccLast 精度等价于：先在逻辑累加格式下按固定顺序计算本条 GEMM 指令对应的点积结果，再将该结果作为最后一项与旧 Acc 值相加。与 Exact 精度不同，AccLast 精度在每条 GEMM 指令内部的每一步乘加后都会舍入到逻辑累加格式，并且在指令边界处保存的累加寄存器值也始终为逻辑累加格式值。

5.2 TACCSET TRS1-T

TACCSET 指令将一个 Tensor Tile 加载到 Acc 寄存器中。对于不同的输入 dtype，其值会被提升到对应的 Acc 逻辑累加格式：

TRS1/TRS2 dtype	Acc 逻辑累加格式
INT8, UINT8, INT16, UINT16	INT32
INT32	INT32
FP8 E4M3/E5M2/E3M4, FP16, BF16	IEEE FP32
FP32, BF32, TF32	IEEE FP32

5.3 TACCGET[.RM3][SAT] TRD-T

TACCGET 指令将 Acc 寄存器逐元素转换格式并写入到 TRD Tensor Tile 中。

可选编码如下：

编码	说明
RM3	数据舍入模式，默认为 RNE
SAT	饱和到目标格式的最大/最小有限值，默认为正常溢出

RM3 编码值如下：

000	RNE	Round to Nearest, ties to Even	默认
001	RTZ	Round Toward Zero	
010	RDN	Round Down	
011	RUP	Round Up	

6. Tensor 算术指令

Tensor Arithmetic 指令负责对 Tensor Tile 执行二元逐元素运算、一元逐元素运算、激活函数和类型转换。该组指令是 Tensor 执行单元的主要计算入口。

6.1 二元逐元素算术指令 TXXX[.M] TRD-T, TRS1-T, TRS2

二元逐元素算术指令对两个源 Tensor 的对应元素执行计算，并写入目的 Tensor。可选 [.M] 启用 Mask 寄存器作为写回选择。部分指令仅对浮点类型或整数类型有效。

指令列表如下：

TADD	$TRD[r][c] = TRS1[r][c] + TRS2[r][c]$	
TSUB	$TRD[r][c] = TRS1[r][c] - TRS2[r][c]$	
TMUL	$TRD[r][c] = TRS1[r][c] * TRS2[r][c]$	
TDIV	$TRD[r][c] = TRS1[r][c] / TRS2[r][c]$	弃用
TMAX	$TRD[r][c] = \max(TRS1[r][c], TRS2[r][c])$	
TMIN	$TRD[r][c] = \min(TRS1[r][c], TRS2[r][c])$	
TAND	$TRD[r][c] = TRS1[r][c] \& TRS2[r][c]$	INT-Only
TOR	$TRD[r][c] = TRS1[r][c] TRS2[r][c]$	INT-Only
TXOR	$TRD[r][c] = TRS1[r][c] \wedge TRS2[r][c]$	INT-Only
TSLL	$TRD[r][c] = TRS1[r][c] \ll TRS2[r][c]$	INT-Only
TSRL	$TRD[r][c] = \text{unsigned}(TRS1[r][c]) \gg TRS2[r][c]$	INT-Only
TSRA	$TRD[r][c] = \text{signed}(TRS1[r][c]) \gg TRS2[r][c]$	INT-Only

二元逐元素算术指令要求 TRS1 TRS2 和 TRD 的 dtype 均一致，唯一的例外是 TMUL 指令，允许的例外组合如下：

TRS1 TRS2 dtype	TRD dtype
INT8	INT16
UINT8	UINT16
INT16	INT32
UINT16	UINT32
FP8 E4M3/E5M2/E3M4	FP16/BF16
FP16/BF16	FP32/TF32

6.2 TMERGE TRD-T, TRS1-T, TRS2

TMERGE 根据 TMask 在两个源 Tensor 之间逐元素选择，写入目的 Tensor:

$$\text{TRD}[r][c] = \text{TMask}[r][c] ? \text{TRS1}[r][c] : \text{TRS2}[r][c]$$

6.3 一元逐元素算术指令 TXXX[.M] TRD-T, TRS1-T

二元逐元素算术指令对源 Tensor 的对应元素执行计算，并写入目的 Tensor。

可选 [.M] 启用 Mask 寄存器作为写回选择。

指令列表如下：

TNOT	$\text{TRD}[r][c] = \sim \text{TRS1}[r][c]$	
TABS	$\text{TRD}[r][c] = \text{TRS1}[r][c] $	
TNEG	$\text{TRD}[r][c] = -\text{TRS1}[r][c]$	
TSQRT	$\text{TRD}[r][c] = \sqrt{\text{TRS1}[r][c]}$	FP-Only, Appr
TRSQRT	$\text{TRD}[r][c] = 1/\sqrt{\text{TRS1}[r][c]}$	FP-Only, Optional, Appr
TEXP	$\text{TRD}[r][c] = \exp(\text{TRS1}[r][c])$	FP-Only, Appr
TRCP	$\text{TRD}[r][c] = 1/\text{TRS1}[r][c]$	FP-Only, Appr
TRELU	$\text{TRD}[r][c] = \max(0, \text{TRS1}[r][c])$	
TRELU6	$\text{TRD}[r][c] = \min(\max(0, \text{TRS1}[r][c]), 6)$	
TSIGMOID	$\text{TRD}[r][c] = 1/(1+\exp(-\text{TRS1}[r][c]))$	FP-Only, Optional, Appr
TTANH	$\text{TRD}[r][c] = \tanh(\text{TRS1}[r][c])$	FP-Only, Optional, Appr
TGELU	$\text{TRD}[r][c] = \text{GELU}(\text{TRS1}[r][c])$	FP-Only, Optional, Appr

一元逐元素算术指令要求 TRS1 和 TRD 的 dtype 均一致。部分指令仅对浮点类型或整数类型有效。Optional 意为非必须实现的指令。Appr 意为允许非精确值实现的指令。

6.4 TCVT[.RM3][.OV][.M] TRD-T, TRS1-T, [max, min]

TCVT 在不同 Tensor dtype 之间执行逐元素类型转换。可选 [.M] 启用 Mask 寄存器作为写回选择。启用 [.RM] 指定舍入策略。启用 [.OV] 指定数值溢出策略。

RM3 编码值如下：

000	RNE	Round to Nearest, ties to Even	默认
001	RTZ	Round Toward Zero	
010	RDN	Round Down	
011	RUP	Round Up	

OV 编码值如下：

000	DEF	整数：按目标位宽截断，溢出取模回绕 浮点：溢出值设置为浮点 Inf	默认
001	SAT	饱和到目标类型可表示的最大/最小有限值	
010	CLIP	饱和到给定的 [min, max] 范围	

7. Tensor Mask 指令

Tensor Mask 指令用于构造、移动、逻辑组合以及比较生成 TMask。TMask 是 32×32 的二维谓词阵列，TMask[r][c] 对应 Tensor 逻辑坐标 [r][c] 的元素。

7.1 TMMV TRD-M-8, TRS1-M-8

TMMV 将一个源操作数复制到目的操作数，仅支持宽度为 8 的数据类型（解释为 bool）或 Mask 寄存器。

7.2 TMNOT TRD-M-8, TRS1-M-8

TMNOT 将一个源操作数复制到目的操作数同时按布尔语义取反，仅支持宽度为 8 的数据类型（解释为 bool）或 Mask 寄存器。

7.3 Mask 二元逻辑指令 TMXXX TRD-M-8, TRS1-M-8, TRS2-M-8

对两个源操作数进行逐元素布尔运算并写入目的操作数，仅支持宽度为 8 的数据类型（解释为 bool）或 Mask 寄存器。

指令列表如下：

指令	说明
TMAND	逻辑与
TMNAND	逻辑与非
TMOR	逻辑或
TMNOR	逻辑或非
TMXOR	逻辑异或
TMNXOR	逻辑同或

7.4 Tensor 比较指令 TMXX TRD-M-8, TRS1-T, TRS2

比较两个 Tensor 源操作数，并将比较结果写入目的操作数。TRS1 和 TRS2 的 dtype 应一致。目的操作数可以是 Mask 寄存器或宽度为 8 bits 的 Tile（解释为 bool）。

指令列表如下：

指令	说明
TMEQ	$D = (A == B)$
TMNEQ	$D = (A != B)$
TMLT	$D = (A < B)$
TMGE	$D = (A >= B)$
TMLE	$D = (A <= B)$
TMGT	$D = (A > B)$

8. Tensor Reduce 指令

Tensor Reduce 指令对一个 Tensor Tile 的元素进行归约计算，并将结果写入目的 Tensor 的指定位置或广播到一组位置。

8.1 TREDXXX[.B][.C|.R][.M] TRD, TRS1, [Row], [Col]

Reduce 指令接受一个源 Tensor TRS1，根配置字段决定归约范围与写回形式，并将结果写入目的 Tensor TRD。Mask 寄存器控制哪些输入元素有效。

可选编码如下：

编码	说明
B	将结果广播到整个输出 Tile，默认为不处理
C	每一列独立归约，默认对整个 Tile 归约
R	每一行独立归约，默认对整个 Tile 归约
M	按照 Mask 使能输入元素，默认为使用全部输入

Row 与 Col 操作数指定了结果写回到目标操作数的位置，仅在没有编码 B 选项的非广播模式下有效，未被写入的位置保持原值。当编码了 C 选项时，Col 操作数无效，结果写回一整行，当与 B 同时使用时，结果在对应列内广播；当编码了 R 选项时，Row 操作数无效，结果写回一整列，当与 B 同时使用时，结果在对应行内广播。

TRED 指令要求 TRS1 与 TRD 的 dtype 必须一致。

指令列表如下：

指令	说明
TREDMIN	$D = \min(\text{TRS1})$
TREDMAX	$D = \max(\text{TRS1})$
TREDSUM	$D = \text{sum}(\text{TRS1})$
TREDAVG	$D = \text{average}(\text{TRS1})$

8.2 TPOLXXX(2|4)[.M] TRD, TRS1, [Row], [Col]

Pooling 指令对源 Tensor 执行固定倍率的局部窗口归约，用于降采样或局部统计。Mask 寄存器控制哪些输入元素有效。

可选编码如下：

编码	说明
2	使用 2×2 pooling
4	使用 4×4 pooling
M	按照 Mask 使能输入元素，默认为使用全部输入

Row 与 Col 操作数指定了结果写回到目标操作数的位置偏移，未被写入的

位置保持原值。

TPOL 指令要求 TRS1 与 TRD 的 dtype 必须一致。

指令列表如下：

指令	说明
TPOLMIN	$D = \min(\text{TRS1})$
TPOLMAX	$D = \max(\text{TRS1})$
TPOLSUM	$D = \text{sum}(\text{TRS1})$
TPOLAVG	$D = \text{average}(\text{TRS1})$

9. Tensor 构造指令

9.1 TFILL TRD, Value, [RowStep=0], [ColStep=0]

TFILL 根据初始值、行步进和列步进构造一个整数型 Tensor Tile：

$$\text{TRD}[r][c] = \text{Value} + \text{RowStep} * r + \text{ColStep} * c$$

RowStep, ColStep 均为通过标量寄存器传递的 16 位整数，其是否带符号位取决于 TRD dtype 是否为有符号整数。

9.2 TFILLLINE TRD, Value, [Step=0]

TFILLLINE 根据初始值、行步进和列步进构造一个整数型 Tensor Tile 。与

TFILL 相比，TFILLLINE 按照行主序或列主序的线性排列计算步长：

$$\text{TRD}[r][c] = \text{Value} + \text{Step} * (r * \text{TWID} + c)$$

Step, ColStep 为通过标量寄存器传递的 16 位整数，其是否带符号位取决于 TRD dtype 是否为有符号整数。

9.3 TBCAST[.R|.C] TRD, TRS1, [Row], [Col]

TBCAST 指令将源 Tensor Tile 中的指定行列或指定元素广播到整个目标 Tensor Tile。Row 和 Col 参数指定了待广播的元素的行列编号。可选编码参数如

下:

编码	说明
R	逐行广播, Row 参数此时无效
C	逐列广播, Col 参数此时无效

9.4 TSHIFT[.Z] TRD, TRS1, [RowOff], [ColOff]

TSHIFT 指令将 Tensor Tile 中的元素按照 Tile 索引进行移动。RowOff 和 ColOff 参数分别指定了行索引和列索引的移动量:

$$\text{TRD}[r][c] = \text{TRS1}[(r + \text{RowOff}) \% \text{THEI}][(c + \text{ColOff}) \% \text{TWID}]$$

特殊的, 当指定了 .Z 编码时, 超出 Tile 范围的元素会被丢弃, 移动导致的空白元素被置 0:

$$\begin{aligned} \text{TRD}[r][c] &= \text{TRS1}[r + \text{RowOff}][c + \text{ColOff}], 0 \leq (r + \text{RowOff}) < \text{THEI}, 0 \leq (c + \text{ColOff}) < \text{TWID} \\ \text{TRD}[r][c] &= 0, \text{ Otherwise} \end{aligned}$$

9.5 TTRANS[.RR|.RC] TRD, TRS1

TTRANS 指令对 TRS1 进行如下三种操作之一并写入 TRD:

TTRANS	转置
TTRANS.RR	Reverse Row, 上下镜像
TTRANS.RC	Reverse Col, 左右镜像

9.6 TSETMASK[.I][.LT|UT|.D] TRD-M-8, Up, Bottom, Left, Right

TSETMASK 指令将一个 Tile 中的指定部分进行布尔填充:

- Up, Bottom, Left, Right 指定了一个矩形子区域在上下左右四个方向上到 Tile 边界的间隔元素数
- 可选的 .LT (Lower-Triangle) 和 .UT (Upper-Triangle) 编码分别选定该矩形子区域中被主对角线划分的下三角和上三角子区域。是否指定 .D (Diagonal) 编码决定了三角子区域是否包含对角线上的元素。

3. 默认的，输出 Tile 中仅选定子区域中的元素为布尔 1，其他元素置 0；

可选的 .I (Invert) 编码会将整个输出 Tile 逐元素布尔取反。

对于选定矩形子区域中的元素，定义其相对矩形子区域左上角的行列编号分别为 lr 和 lc ，上下三角子区域的定义为：

TSETMASK.UT	$V[lr][lc] = (lc > lr)$
TSETMASK.UT.D	$V[lr][lc] = (lc \geq lr)$
TSETMASK.LT	$V[lr][lc] = (lc < lr)$
TSETMASK.LT.D	$V[lr][lc] = (lc \leq lr)$

10. Tensor 控制指令

10.1 TFENCE[.R] rd, [TRS1], Timeout

TFENCE 指令在标量流水线中阻塞，等待当前 hart 上已经提交到 Tensor 执行单元的所有 Tensor 指令完成。Timeout 参数指定了超时周期数，当成功等待所有 Tensor 指令完成时，指令在标量寄存器 rd 返回 1；反之，当因超时停止等待时，在标量寄存器 rd 返回 0。Timeout 不能为 0 以避免无限等待。

当指定 .R 编码时，TRS1 有效，TFENCE.R 指令等待所有以 TRS1 为目标操作数的 Tensor 指令。

10.2 TCANCEL

TCANCEL 取消当前 hart 上所有 on-fly Tensor 指令，并复位 Tensor 执行单元内部状态。TCANCEL 执行后，Tensor 寄存器内容处于未指定状态。

11. Tensor FIFO 指令（草稿）

Tensor IPC 指令用于 hart 之间通过 Tensor FIFO 传递一个 Tensor Tile。该机制用于多核 TPU 中的局部 Tensor 通信，支持一对一发送和接收。包含发送指令 TSEND 和接收指令 TREC V。TSEND 将本地 Tensor Tile 发送到目标 hart 的接收 slot，TREC V 从本地接收 slot 接收 Tensor Tile 并写入 Tensor 寄存器。

每个 hart 拥有最多 8 个接收 slot，可从 CSR tslotnum 获取。Tensor FIFO 的流程如下：

1. TSEND 执行并阻塞，直到目标 hart 的目标 slot 成功接收并返回 ACK
2. TREC V 执行并阻塞，直到本地的目标 slot 中存在完成 Tensor Tile
3. TREC V 成功接收时，向来源 hart 发送 ACK

Tensor FIFO 指令不提供额外内存同步语义，TSEND 和 TREC V 指令的顺序和对应关系应当由标量区通过 AMO 指令等机制保证。Tensor FIFO 指令仅保证其执行顺序严格按照指令被提交到 Tensor 执行单元的顺序。

TODO：未来需要支持多任务下 **Tensor FIFO** 指令的上下文切换。

11.1 TSEND TRS1, HartID, SlotID

TSEND 将本 hart 的一个 Tensor 操作数发送到目标 hart 的指定 slot。

11.2 TREC V TRD, SlotID

TREC V 从本 hart 的指定 slot 接收一个 Tensor，并写入目的 Tensor 操作数。